

Naproche Natural Language Formalizations with LLMs

Peter Koepke and Joshua Wirtz

Mathematical Institute
University of Bonn, Germany

11th Conference on Artificial Intelligence and Theorem Proving (AITP 2026)
September 3, 2026

Naproche Example: The Cantor–Schröder–Bernstein Theorem

Theorem

Let x, y be sets. x and y are equinumerous iff there exists a injective map from x to y and there exists an injective map from y to x .

Proof.

[...] Consider an injective map f from x to y and an injective map g from y to x . [...] Define $H(u) := x \setminus g[y \setminus f[u]]$ for $u \in \mathcal{P}(x)$. [...] H has a fixed point. Indeed x is a set and H is a map from $\mathcal{P}(x)$ to $\mathcal{P}(x)$ that preserves subsets. Consider a fixed point X of H . [...] Define

$$h(a) := \begin{cases} f(a) & : a \in X \\ g^{-1}(a) & : a \notin X \end{cases}$$

for $a \in x$. [...] Then h is a bijection between x and y . □

Naproche: Natural Proof Checking

Formal Theory
Language
(ForTheL)



Calculus of
Text Correctness
(CTC)



Automatic
Theorem Provers
(ATPs)



*Approximates informal
mathematical text*

Approximates proofreading of human mathematicians

Why Naproche + LLMs?

-  **Natural interface:** Informal $\rightarrow$ ForTheL
-  **Natural proving:** *Intelligent proofreading* handles low-level justifications
-  **Natural output:** Formalizations read like textbook mathematics
-  **Data advantage:** ForTheL resembles ordinary mathematical literature

What Makes a *Good* Naproche Formalization?

Formal Criteria

- Σ ForTheL validity
- $\top$ $\text{\LaTeX}$ validity
- ($\times$) Logical correctness
-  Ontological correctness
- ...

Qualitative Criteria

-  Readability
-  Faithfulness
-  Theory reuse
-  Exposition
- ...

From *interactive* Chats to *autonomous* Agents

- A year ago: Formalization of Euclid's Lemma from Wikipedia in Naproche with ChatGPT (GPT-5 Thinking)
- Started from small axiomatization of addition and multiplication on $\mathbb{N}$
- Produced a *good* Naproche formalization
- 🔗 Required hours of tedious prompting and copy-pasting
- 🚧 Human bottleneck between LLM and Naproche

🏠 Closing the Loop

Agents can *edit* the formalization, *run Naproche*, inspect errors, and *iterate autonomously*.

Experimental Setup



Agentic System

- Visual Studio Code
- GitHub Copilot Chat
- GPT-5.3-Codex Xhigh
- Agent mode



Environment

- Isabelle2025-02
- Naproche examples
- `verify.sh` script



Target

Simple theorems from Freek
Wiedijk's list of 100
theorems

Procedure

1. Removed existing formalizations of target
2. Issued *single* task prompt
3. Zero human intervention: agent determined task completion

Autoformalizing Euclid's Lemma from Wikipedia

▢ Prompt

*Formalize Euclid's lemma in Naproche. The formalization must follow the **proof by induction from Wikipedia**. The formalization must be **self-contained**. The formalization must be written to a single file named `euclids_lemma.ftl.tex`. The formalization must **successfully verify** when running the command `./verify.sh`.*

🔗 Research Questions

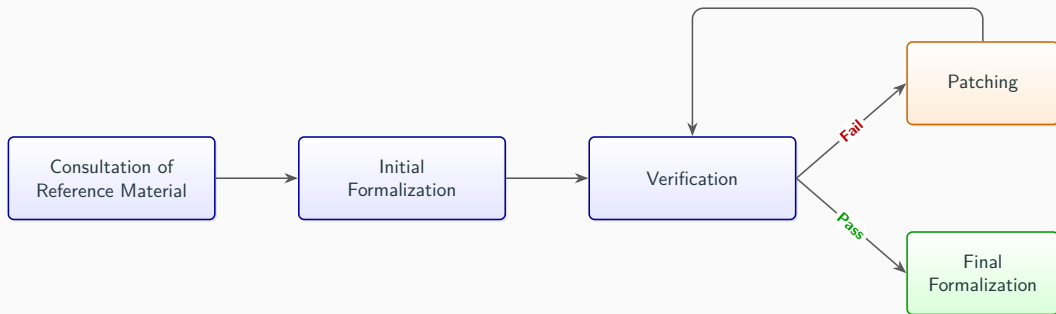
🔗 Formalization workflow

🔗 Error handling

🔗 Theory and proof construction

🎯 Faithfulness to Wikipedia proof

The Agent's Formalization Workflow



Evaluation of Behavior

- Literal search patterns
n|ab, verbatim Wikitext
- 12 verify-patch iterations
- Overcame all parser and reasoner errors
despite sometimes cryptic error messages
- Patches sometimes destructive
- Removed Euclid's Lemma
leaving only Generalized Euclid's Lemma

Evaluation of Formalization

- ≈ 9 min
- Correctly stated Generalized Euclid's Lemma (for $\mathbb{N}$)
- Structurally follows Wikipedia induction proof
- Handled omitted boundary case
- Foundational theory copied verbatim from examples
 $\mathbb{N}$ instead of $\mathbb{Z}$
- Insufficient algebraic infrastructure
- Central algebraic steps bypassed with axioms
- **Critical failure:** One Axiom contradicts the standard model

Autoformalizing a Proof of the Cantor–Schröder–Bernstein Theorem

▢ Prompt

Fill in the missing proof inside the Cantor–Schroeder–Bernstein formalization at `archive/articles/source/Cantor–Schroeder–Bernstein.ftl.en.tex`. The proof must successfully verify when running the command `./verify.sh`.

Theorem (Cantor–Schröder–Bernstein)

Let x, y be sets. x and y are equinumerous iff there exists a injective map from x to y and there exists an injective map from y to x .

- Read imported formalizations
- Searched for utilizes macros
- 10 verify–patch iterations
- Only syntactical errors
- Local, surgical fixes



Formalization Plan

1. $H: \mathcal{P} \rightarrow \mathcal{P}$
2. H preserves subsets
3. H has a fixed point
4. h is surjective
5. h is injective
6. h is a bijection

Evaluation of Formalization

- ≈ 12 min
- Uses provided Knaster–Tarski theorem
- Follows Schröder's "The fixed point property for ordered sets" suggested in surrounding prose
- Easy to read

Generated Proof

[...] Consider an injective map f from x to y and an injective map g from y to x . [...] Define $H(u) := x \setminus g[y \setminus f[u]]$ for $u \in \mathcal{P}(x)$. [...] H has a fixed point. Indeed x is a set and H is a map from $\mathcal{P}(x)$ to $\mathcal{P}(x)$ that preserves subsets. Consider a fixed point X of H . [...] Define

$$h(a) := \begin{cases} f(a) & : a \in X \\ g^{-1}(a) & : a \notin X \end{cases}$$

for $a \in x$. [...] Then h is a bijection between x and y .

-  Agents can handle niche, prototype systems
-  Unguided autoformalization is unreliable
-  Formal verification is not enough
-  Natural formalizations simplify human review